

Below is a **complete, instructor-style, code-first relationship setup** followed by a **fully working mini-project** you can copy and run.

This includes:

1. Entities with relationships
2. DbContext configuration
3. Code-first migrations
4. CRUD API using ASP.NET Core Web API
5. Eager & Lazy Loading demonstration
6. Sample seed data

The solution uses a **One-to-Many** relationship (Department → Employees).

1. Project Structure (ASP.NET Core Web API)

EFCoreDemo/

├─ Controllers/

| └─ DepartmentsController.cs

| └─ EmployeesController.cs

├─ Data/

| └─ AppDbContext.cs

├─ Models/

| └─ Department.cs

| └─ Employee.cs

└─ Program.cs

└─ EFCoreDemo.csproj

2. Step 1: Create Models (Code-First Entities)

Models/Department.cs

```
using System.Collections.Generic;
```

```
namespace EFCoreDemo.Models
```

```
{  
    public class Department  
    {  
        public int DepartmentId { get; set; }  
        public string Name { get; set; }  
  
        // ONE department has MANY employees  
        public virtual ICollection<Employee> Employees { get; set; } = new List<Employee>();  
    }  
}
```

Models/Employee.cs

```
namespace EFCoreDemo.Models
```

```
{  
    public class Employee  
    {  
        public int EmployeeId { get; set; }  
        public string FullName { get; set; }  
  
        // Foreign Key  
        public int DepartmentId { get; set; }  
  
        // Navigation Property  
        public virtual Department Department { get; set; }  
    }  
}
```

```
}  
}
```

Note:

Using virtual allows **Lazy Loading** when proxies are enabled.

3. Step 2: Create DbContext

Data/AppDbContext.cs

```
using EFCoreDemo.Models;  
using Microsoft.EntityFrameworkCore;  
  
namespace EFCoreDemo.Data  
{  
    public class AppDbContext : DbContext  
    {  
        public AppDbContext(DbContextOptions<AppDbContext> options) : base(options) {}  
  
        public DbSet<Department> Departments { get; set; }  
        public DbSet<Employee> Employees { get; set; }  
  
        protected override void OnModelCreating(ModelBuilder modelBuilder)  
        {  
            // Department → Employees (One-to-Many)  
            modelBuilder.Entity<Department>()  
                .HasMany(d => d.Employees)  
                .WithOne(e => e.Department)  
                .HasForeignKey(e => e.DepartmentId)  
        }  
    }  
}
```

```

        .onDelete(DeleteBehavior.Cascade);

// Seed some data
modelBuilder.Entity<Department>().HasData(
    new Department { DepartmentId = 1, Name = "IT"},
    new Department { DepartmentId = 2, Name = "Accounts"}
);

modelBuilder.Entity<Employee>().HasData(
    new Employee { EmployeeId = 101, FullName = "Ravi Kumar", DepartmentId = 1 },
    new Employee { EmployeeId = 102, FullName = "Meena Sharma", DepartmentId = 1 },
    new Employee { EmployeeId = 201, FullName = "Arjun Patel", DepartmentId = 2 }
);
}
}
}

```

4. Step 3: Modify Program.cs to Register EF Core

Program.cs

```

using EFCoreDemo.Data;

using Microsoft.EntityFrameworkCore;

var builder = WebApplication.CreateBuilder(args);

// Enable Lazy Loading (optional)
builder.Services.AddDbContext<AppDbContext>(options =>

```

```

options.UseLazyLoadingProxies()
    .UseSqlServer(builder.Configuration.GetConnectionString("DefaultConnection")));

builder.Services.AddControllers();
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();

var app = builder.Build();

if (app.Environment.IsDevelopment())
{
    app.UseSwagger();
    app.UseSwaggerUI();
}

app.UseAuthorization();

app.MapControllers();

app.Run();

```

5. Step 4: Add Connection String in appsettings.json

```

{
  "ConnectionStrings": {
    "DefaultConnection":
"Server=localhost;Database=EFCoreDemoDb;Trusted_Connection=True;TrustServerCertificate=True"
  }
}

```

```
}  
}
```

6. Step 5: Create Code-First Migrations

Run these commands in Package Manager Console:

```
Add-Migration InitialCreate
```

```
Update-Database
```

Database is now created with seed data.

7. Step 6: Create Controllers

Controllers/DepartmentsController.cs

```
using EFCoreDemo.Data;
```

```
using EFCoreDemo.Models;
```

```
using Microsoft.AspNetCore.Mvc;
```

```
using Microsoft.EntityFrameworkCore;
```

```
namespace EFCoreDemo.Controllers
```

```
{
```

```
    [ApiController]
```

```
    [Route("api/[controller]")]
```

```
    public class DepartmentsController : ControllerBase
```

```
    {
```

```
        private readonly AppDbContext _context;
```

```
        public DepartmentsController(AppDbContext context)
```

```
        {
```

```
    _context = context;  
}
```

```
// Get all departments with employees (Eager Loading)
```

```
[HttpGet("eager")]
```

```
public async Task<IActionResult> GetDepartmentsEager()
```

```
{
```

```
    var data = await _context.Departments
```

```
        .Include(d => d.Employees)
```

```
        .ToListAsync();
```

```
    return Ok(data);
```

```
}
```

```
// Lazy loading version
```

```
[HttpGet("lazy")]
```

```
public async Task<IActionResult> GetDepartmentsLazy()
```

```
{
```

```
    var data = await _context.Departments.ToListAsync();
```

```
    // Employees will load only when accessed here
```

```
    foreach (var d in data)
```

```
    {
```

```
        var empCount = d.Employees.Count;
```

```
    }
```

```
    return Ok(data);
```

```

    }

//explicit loading
[HttpGet("explicit/{id}")]
public async Task<IActionResult> GetDepartmentExplicit(int id)
{
    // Load only the department
    var department = await _context.Departments
        .FirstOrDefaultAsync(d => d.DepartmentId == id);

    if (department == null)
        return NotFound();

    // Explicitly load Employees
    await _context.Entry(department)
        .Collection(d => d.Employees)
        .LoadAsync();

    return Ok(department);
}

// Create a new Department
[HttpPost]
public async Task<IActionResult> CreateDepartment(Department department)
{
    _context.Departments.Add(department);
    await _context.SaveChangesAsync();
    return Ok(department);
}

```



```
    }  
    }  
}
```

Controllers/EmployeesController.cs

```
using EFCoreDemo.Data;  
using EFCoreDemo.Models;  
using Microsoft.AspNetCore.Mvc;  
using Microsoft.EntityFrameworkCore;  
  
namespace EFCoreDemo.Controllers  
{  
    [ApiController]  
    [Route("api/[controller]")]  
    public class EmployeesController : ControllerBase  
    {  
        private readonly AppDbContext _context;  
  
        public EmployeesController(AppDbContext context)  
        {  
            _context = context;  
        }  
  
        // Get employees with department (Eager)  
        [HttpGet("eager")]  
        public async Task<ActionResult> GetEmployeesEager()
```

```

{
    var data = await _context.Employees
        .Include(e => e.Department)
        .ToListAsync();

    return Ok(data);
}

// Create employee
[HttpPost]
public async Task<IActionResult> CreateEmployee(Employee employee)
{
    _context.Employees.Add(employee);
    await _context.SaveChangesAsync();
    return Ok(employee);
}
}
}

```

8. Step 7: Run and Test API

After running the project, open Swagger UI:

<https://localhost:5001/swagger>

You will see:

- GET: /api/Departments/eager
- GET: /api/Departments/lazy
- GET: /api/Employees/eager
- POST: /api/Employees

- POST: /api/Departments
-

9. Demonstration of Loading Behavior

Eager Loading Example – SQL Generated

SELECT * FROM Departments d

LEFT JOIN Employees e ON d.DepartmentId = e.DepartmentId;

Lazy Loading Example – SQL Generated

1. Load departments
2. Access department.Employees → triggers separate SQL query for each department

This clarifies the difference between the two strategies.

10. Summary of What You Received

You now have:

1. Complete code-first relationship configuration
2. Fully working Web API sample
3. Eager loading example
4. Lazy loading example
5. Seed data included
6. Ability to CRUD Departments and Employees